

YAMI (Yet Another Mixer)

piotr@majdak.com, michael.mihocic@oeaw.ac.at

Acoustics Research Institute, Austrian Academy of Sciences, Vienna, Austria

Last update: 17.03.2020 by Michael Mihocic

YAMI is a mixer, sampler, recorder, synthesizer and real time filtering device for multi-channel audio. Coming without any user interface, YAMI can be controlled via network by an application independently running anywhere in network.

Overview

The ExpSuite graphical user interface is responsible for the interaction with the user and the control of the processes. Real-time data processing and the interface to the sound card are part of the YAMI module (Yet Another Mixer, implemented in pd). It is responsible for playback, recording, saving, generating, real-time filtering of audio signals and the connection to the tracking system.

The communication between the ExpSuite framework and YAMI works via UDP with OSC (Open Sound Control) as its protocol. The usage of UDP allows a transfer to a different computer to avoid high CPU load on the control system. OSC uses a standardized transmitting syntax that was realized on multiple platforms so far.

YAMI connects to a defined sound interface via ASIO and allows to control all channels simultaneously, with low latency time (max. 40ms).

Functionality

YAMI contains of many modules, which support different functionality:

- Play module: play a WAV file to a stream
- DAC module: pass a main stream and an additional stream to output device
- VBAP module (vector base amplitude panning): playback on a large number of audio channels
- BVA module (binaural virtual acoustics): output with Spatializer (2 channels)
- ADC module: get audio data from input device and pass to a stream
- Record module: record a stream to buffer and save it to a WAV file
- Synth module: synthesize a stream (cosine, white and pink noise supported)
- Control module: control the behaviour of YAMI
- Head Tracker module: initialise and control head tracker using com port, collect, save or send data to client
- Filter module: filter an audio stream according to changing parameters in real time. Filter switching or interpolation algorithms not specified yet.

(for more details on modules see illustrations at the end of document)

All modules can be combined in any way (but don't try to connect two output streams together ;-).

There is a different amount of modules in YAMI, depending on the used YAMI version:

- 2/24/100 Play modules to stream 2/24/100 files simultaneous
- 2/24/100 DAC modules to use 2/24/100 channels of audio (digital-analog conversion)
- 2/4/16 ADC modules to get 2/4/16 channels of audio (analog- digital conversion)
- 2/4/16 Record modules to record 2/4/16 channels simultaneous

- 2 Synth modules to synthesize two independent streams
- Sequencer module to put a sequence of commands with accurate delay in between

See the following table for the number of channels in YAMI v1.1 implemented patches:

Version	Play	DAC	ADC	Record	Synth
YAMI2	2	2	2	2	2
YAMI	24	24	4	4	2
YAMI16adc	24	24	16	16	2
YAMI100	100	100	2	4	2

Connected to a network by UDP, YAMI understands a simple set of command using OSC (Open Sound Control) and can be controlled from a remote machine to provide a scalable audio solution. A response channel can be established to get a response from YAMI, using UDP/OSC as well.

Details on the network connection:

- UDP, Port 10003
- Response channel: UDP, Port user defined by control command

In OSC all commands use URL-like syntax (see OSC specifications for details) e.g.:

- /Control/Version
- /Control/Response connect localhost 10003
- /Play/SetFade 30

This syntax is very useful implementing independent modules, what leads to very easy debugging and fast development.

Syntax of commands

/Control: all control commands

- /Response connect address port: establish a response connection using address/port
- /Response disconnect: disconnect from the response channel
- /Response send XXX: send XXX through the response channel
- /Version: send version ID
- /SampleRate: get the sample rate in Hz
- /CPULoad: get the CPU load in %
- /LoadObject NAME OBJ: load dynamically an object (or abstraction) to pd. NAME is an arbitrary name of the object and OBJ is the file name. Specifying OBJ without any path, OBJ must be in the same directory as YAMI patches.
- /DelObject NAME: delete object NAME.

/DAC: pass streams to audio output

- /SetStream/dacX set Y: connect DAC-channel dacX to a stream Y; dacX=*:all channels
- /SetStream/dacX set silence: disconnect DAC channel dacX from any stream (play silence); dacX=*:all channels
- /SetVol/dacX INT: set volume for DAC channel dacX to INT. INT=0: mute, INT=100: full; dacX=*:all channels
- /SetAddStream/dacX set streamY: connect additional input of a DAC-channel dacX to a stream
- /SetAddStream/dacX set silence: disconnect the additional input of a DAC channel dacX

Allowed streams for the DAC module: adcX, playX

/Play: play stream from a WAV file

- /SetFade INT: set fade in/out time in ms for all channels
- /OpenWAV/Y open XXX: open WAV-file XXX for stream playY, XXX must be local file name; Y=*: all streams
- /Start/Y: play stream playY; Y=*: all streams
- /Stop/Y: stop stream playY; Y=*: all streams
- /StartSynced/Y: start recording with synchronized playing (corresponds to /Play/Start/Y and /Record/Start/*)
- /StartSynced/Y/stop: stop pending delayed synced playbacks; Y=*: all streams
- /SetDelay/Y INT: set a delay of the begin of playing for channel Y, INT in ms. Must be quantized to 64 samples to achieve full synchronization between all channels. Due to truncation of INT, a float value must be a little bit bigger (0.05ms) than the exact value.
- /StartAll/0: play all streams and start recording using /Rec/Start/*. This is a direct method to start all streams simultaneously.

/ADC: get streams from audio input

- /SetAttn/adcx INT: set volume (attenuation) for ADC channel X to INT. INT=0: mute, INT=100: full; adcX=*:set for all channels
- /GetLevel/adcx: get current RMS level from channel X, response=100: full scale

/Rec: record a stream and save to a WAV file

- /SetStream/recX set Y: connect stream Y to the recording unit recX
- /SetStream/recX set silence: disconnect recording unit recX from any stream
- /SetDur/recX INT: set duration of recording for the rec unit X to INT [ms]
- /Start/recX: start recording of unit recX; recX=*:all units
- /StartSynced/recX: start recording with synchronized playing (corresponds to /Play/Start/* and /Record/Start/recX simultaneously)
- /Stop/recX: stop recording of unit recX; recX=* all units
- /SaveWAV/recX XXX: save recorded data of unit recX (one channel = mono) to file XXX
- /SaveWAV2/recX XXX: save recorded data of units recX and recX+1 (stereo) to file XXX
- /SaveWAV4/recX XXX: save recorded data of unit recX to recX+3 (4 channels) to file XXX

Allowed streams for input of Rec: adcX, playX

/Synth: synthesize a stream with defined signal

- /SetVol/synX FLOAT: set volume for synthesizer unit synX (INT=100: full, 0: disabled)
- /SetLowCut/synX INT: set low cut freq. of high pass filter for unit synX, INT in Hz
- /SetHigCut/synX INT: set high cut freq. of low pass filter for unit synX, INT in Hz
- /SetPar1/synX FLOAT: set parameter 1 for unit synX
- /SetSignal/X STRING: set signal source of unit, possible values:
 - white: white noise, no additional parameters
 - pink: pink noise, no additional parameters
 - cosine: cosine, Par1: frequency in Hz
 - lp4white: 4th order butterworth low pass filtered white noise, Par1: cut off frequency in Hz.
 - lp16white: 16th order butterworth low pass filtered white noise, Par1: cut off frequency in Hz.

/Sequencer: record and play a sequence of commands

There are no more submethods for module Sequence. All commands must be sent as parameters

(/Sequencer command)

- clear: clear the buffer
- add root CMD: add command CMD to the buffer without any delay (useful for the first command in the buffer)
- add X root CMD: add command CMD to the buffer, play X ms the previous command.
- read XXX: read a text file XXX to the buffer
- write XXX: write the buffer to the file XXX
- command without any parameters: play the buffer. The buffer won't be cleared after playing.

/Tracker: control trackers via serial port

- /Load: Abstraction with the functionality must be loaded before using any other commands
- /Unload: unload the tracker abstraction and free the resources
- /InitCOM COMPORT BAUDRATE: initialize a serial port COMPORT [0...3 for serial port 1...4] and establish connection to the head tracker using BAUDRATE [bit per second]. The settings must be according to the hardware configuration (dip switches). All data streams will be stopped.
- /GetFoBStatus: Get the status of tracker:
 - On, but not initialized. Response: /FoBStatus 161 160 0 0 0 0 0 0 0 0 0 0
 - Running in stand alone mode. Response: /FoBStatus 225 160 0 0 0 0 0 0 0 0 0 0
 - Running in Master-Slave mode. Response: /FoBStatus 225 224 0 0 0 0 0 0 0 0 0 0Don't use this command after /InitFoB, because the Tracker will be in the data stream mode after that. You can stop the stream mode using /InitCOM
- /InitFoB S DELAY RATE SCALE: initialize birds and start streaming data.
 - S: number of motion sensors (birds)
 - DELAY: delay necessary to initialize FoB without errors in ms.
 - RATE: repetition rate in stream mode, [1, 2, 3, 4] for 1, 2, 8 or 32 cycles. See table below for the effective streaming rate.
 - SCALE: maximal scaling position: [36, 72] inches.
 - Flock of birds must be in a master-slave configuration. Configuration: "FBB Auto Config. Standalone" or "FBB Auto Config. Master/Slave"
 - /InitFoB responses with /InitFoB after the given delay, which is necessary to initialize FoB without errors. 1st initialization (GetFoBStatus: On) needs 13s. Every initialization after that takes at least 3s.

	<i>Data rate [values/s]</i>	
<i>RATE</i>	<i>Master</i>	<i>Master/Slave</i>
1	412	206
2	206	103
3	51.5	25.75
4	12.875	6.4375

- /SetCurrentValues/S X Y Z A E R: set current position and orientation of the sensor S to coordinate X, Y, Z and azimuth A, elevation E and roll R, S: [0,1]. Range of coordinates: -182.88 to +182.874. Range of angles: -180 to 179.99. Other values cause no errors but are out of range
- /GetCurrentValues/S: get current position X, Y, and Z and orientation azimuth A, elevation E, and roll R for sensor S [0,1]. Response: /GetCurrentValues/S X Y Z A E R
- /TrackMinMaxValues/S Xmin Xmax Ymin Ymax Zmin Zmax Amin Amax Emin Emax Rmin Rmax: track the data and send a message if a value exceeds given threshold for sensor S [0,1].

Response: /TrackMinMaxValues/S DIM, DIM: [Xmin, Xmax, Xinrange, Ymin, Ymax, Yinrange, Zmin, Zmax, Zinrange, Amin, Amax, Ainrange, Emin, Emax, Einrange, Rmin, Rmax, Rinrange]. The response will be sent every time when min/max value is passed (leaving or re-entering range).

Ranges:

- A, R: -180°...-90°...0°...90°...180°(≈ -180°)
- E: -0°...-90°...0°...90°...0°(≈ -0°)
- /DontTrackMinMaxValues/S: Set off comparing mode
- /Simulation/x: x:1...simulate a tracker, 0...don't simulate, tracker must be physically available. Simulation available for /InitFoB, /GetCurrentValues (response in random values), /SetCurrentValues, /Evaluate. Not available for /Buffer commands.
- /Buffer: Save data to a buffer for further analysis:
 - /SetDur X: set the duration of buffers to X samples. Default: 5000 values.
 - /Start: start recording data to the buffer. Buffer is cleared before this command.
 - /Stop: stop recording data to the buffers
 - /SaveWAV/S XXX: save data from buffers of sensor S to WAV file XXX (6-channel file: X, Y, Z, azimuth, elevation and roll), S: 0,1
 - /GetMinVal/S: get minimum values from buffer for X, Y, Z, azimuth, elevation, and roll of sensor S [0,1]. Response: /GetMinVal/S Xmin Ymin Zmin Amin Emin Rmin
 - /GetMaxVal/S: get maximum value from buffer for X, Y, Z, azimuth, elevation, and roll of sensor S [0,1]. Response: /GetMaxVal/S Xmax Ymax Zmax Amax Emax Rmax
 - /GetAvgVal/S: get average value from buffer for X, Y, Z, azimuth, elevation, and roll of sensor S [0,1]. Response: /GetAvgVal/S Xavg Yavg Zavg Aavg Eavg Ravg
- /SendData/S: Send tracker data to another OSC client. S: sensor number [0,1]:
 - /Interval INT: set the interval of sending data [samples]
 - /Connection connect ADDR P: send data to host given by name or IP address ADDR and port P. Data will be sent in interval given by /Interval command.
 - /Connection disconnect: stop sending data.
 - Data will be sent in format: /Tracker/Data/S X Y Z A E R, S: sensor number [0,1].

/Tracker creates streams for the X position trSX, Y position trSY, Z position trSZ, azimuth trSA, elevation trSE and roll trSR, where S is the sensor number [0,1]

Applications with /Tracker:

- HRTF-measurement: record orientation of the head during the system identification and provide the minimum and maximum deviation from the reference position.
- Synthesis of virtual stimuli: send a stream for a direction to real time HRTF-filter
- Controlling position in an experiment: provide a message to experimental software if any a sensor gets out of a given corridor

/MIDI: send and receive data via MIDI

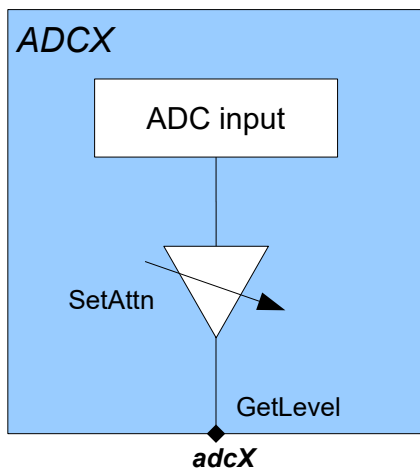
- CH CTRL VAL: send value VAL via MIDI to controller CTRL via channel CH

Any MIDI activity will be sent via response channel using the syntax:

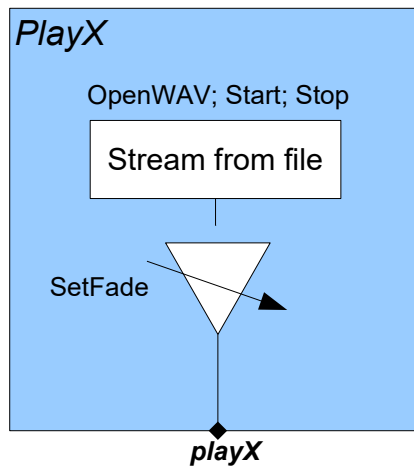
- MIDI CH CTRL VAL, where CH is the channel, CTRL is the controller and VAL is the value of the MIDI event.

YAMI: Modules with output streams

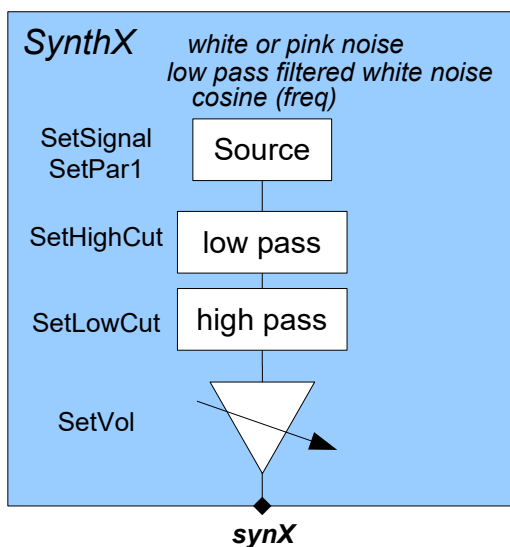
Stream from ADC:



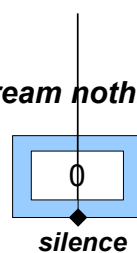
Stream from file:



Synthesize a stream:



Stream nothing:



Legend:

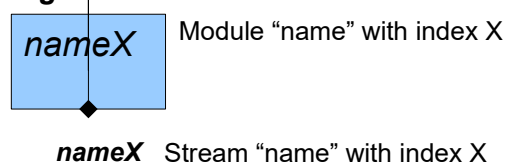
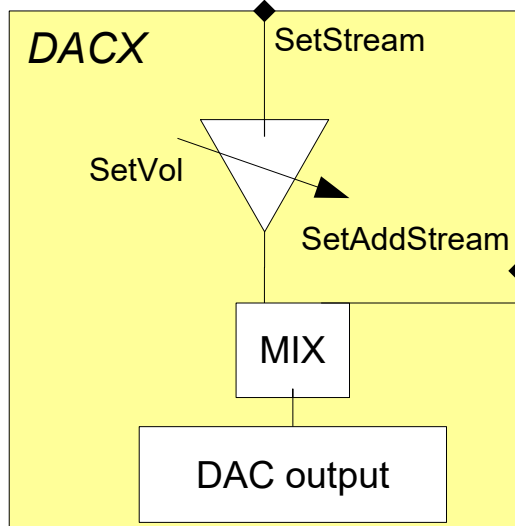


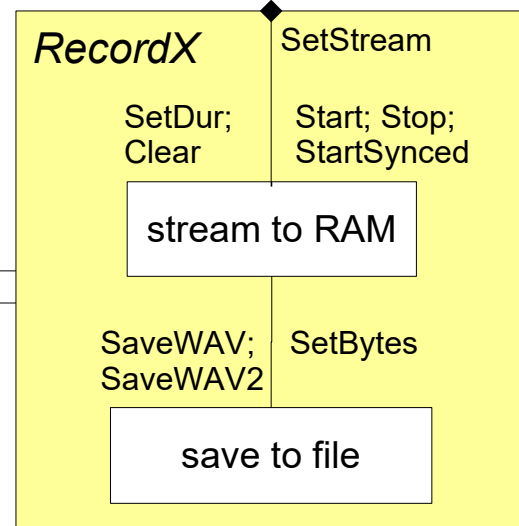
Illustration 1: Output modules

YAMI: Modules processing streams

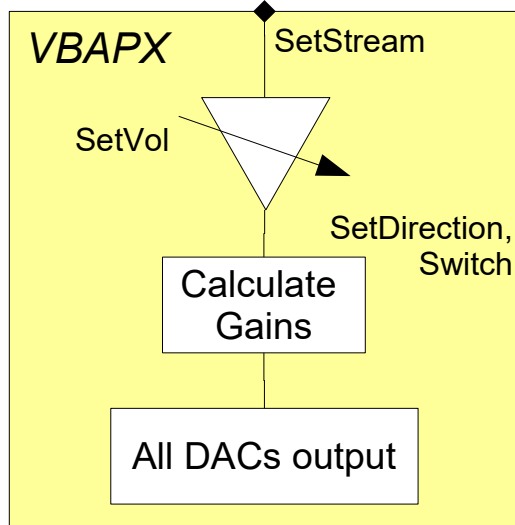
Send stream to DAC:



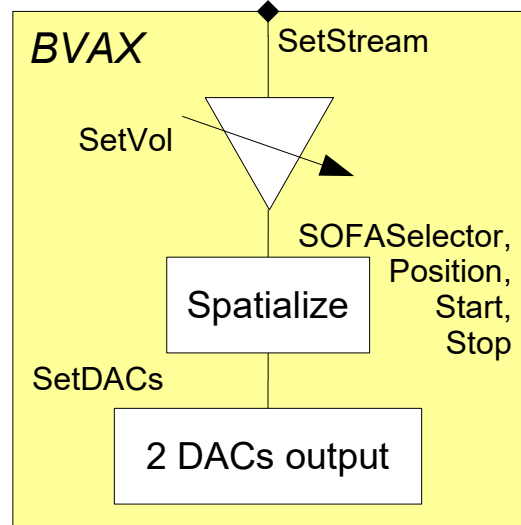
Record stream to file:



Send stream to VBAP:



Send stream to BVA:



Legend:

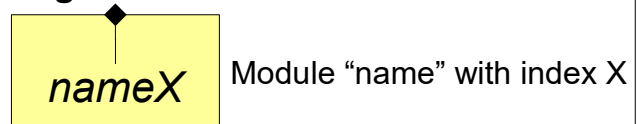
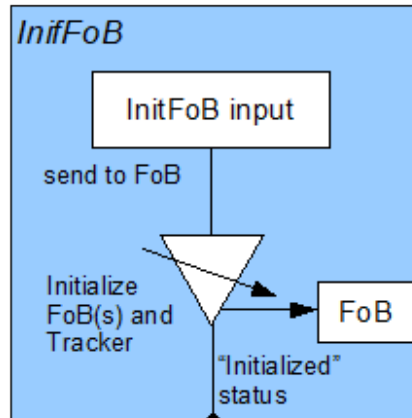


Illustration 2: Input modules

YAMI: Tracker

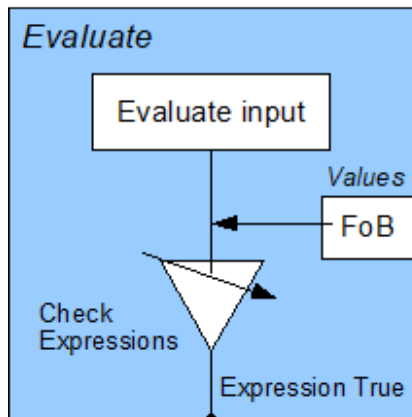
Modules with output streams

Stream from InitFoB:



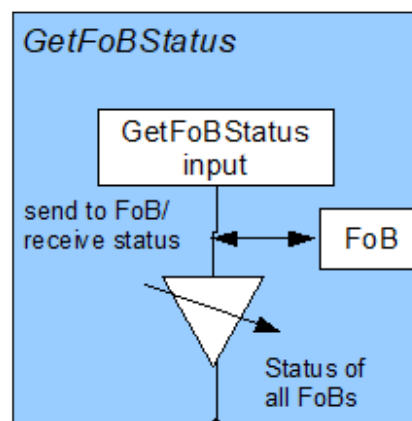
Tracker Initialized

Evaluate



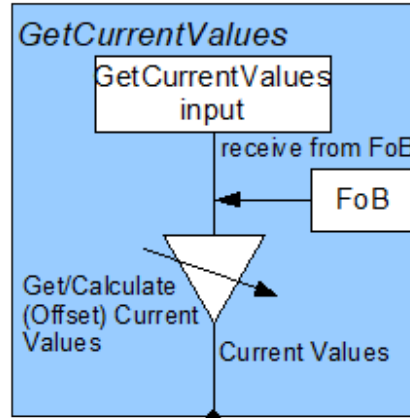
Expression True

GetFoBStatus



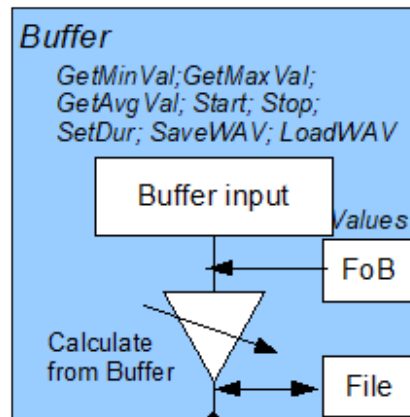
GetFoBStatus ffffffffffffffff

Stream from GetCurrentValues:



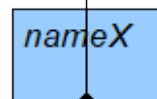
S x y z A E R

Buffer



MinVal; MaxVal; AvgVal

Legend:



Module "name" with index X

nameX Stream "name" with index X

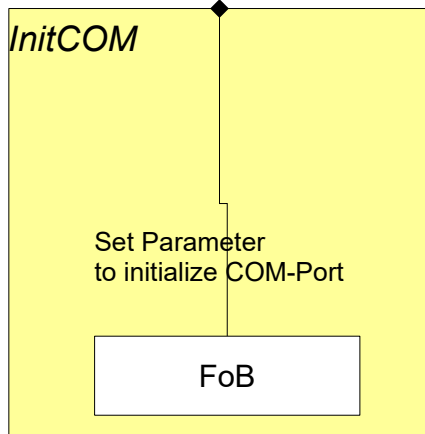
Illustration 3: Tracker modules with output streams

YAMI: Tracker

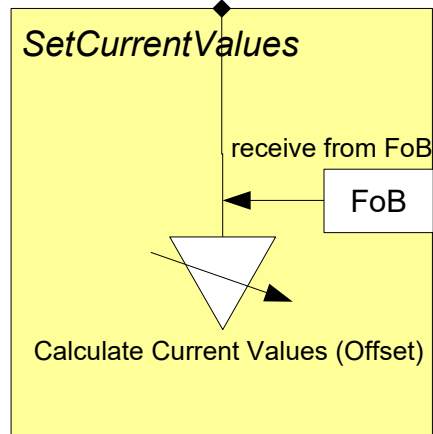
Modules processing streams

Set Parameter
to initialize COM-Port

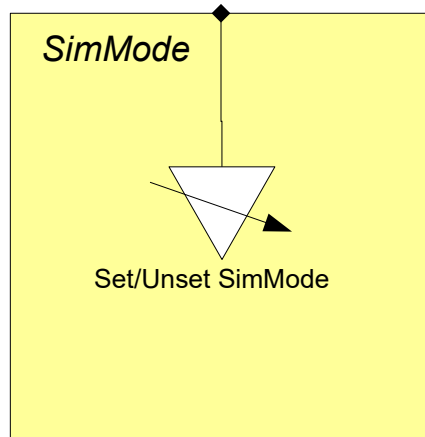
Init Com



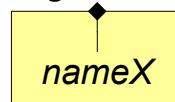
SetCurrentValues



SimMode



Legend:



Module "name" with index X

Illustration 4: Tracker modules processing streams